



POC Avantages Sportifs

Pipeline ETL bout-en-bout — Sport Data Solution

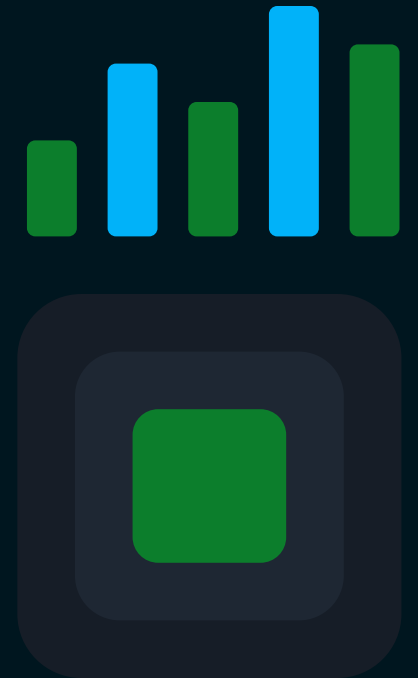
Kestra

PostgreSQL

Python

Great Expectations

Docker



Sommaire

01

Contexte & mission

deux avantages, trois objectifs

02

Architecture

trois couches, un orchestrateur

03

Pipeline & qualité

sept étapes, trois lignes de défense

04

Sécurité & données personnelles

RGPD, pseudonymisation

05

Monitoring & résultats

traçabilité, impact financier

06

Démo & industrialisation

live + passage en production

01

/ Contexte & mission

Deux avantages sportifs à chiffrer avant d'industrialiser.

01 / Contexte

La mission confiée par Juliette

01 / Contexte

La mission confiée par Juliette

Encourager la pratique sportive des 161 salariés via deux avantages — et en mesurer le coût.

01 / Contexte

La mission confiée par Juliette

Encourager la pratique sportive des 161 salariés via deux avantages — et en mesurer le coût.

PRIME

+5 % du salaire brut

Pour les salariés venant au bureau en mode actif (marche, course, vélo, trottinette). Vérifié par la distance domicile-travail.

01 / Contexte

La mission confiée par Juliette

Encourager la pratique sportive des 161 salariés via deux avantages — et en mesurer le coût.

PRIME

+5 % du salaire brut

Pour les salariés venant au bureau en mode actif (marche, course, vélo, trottinette). Vérifié par la distance domicile-travail.

BIEN-ÊTRE

5 jours offerts / an

Pour les salariés réalisant au moins 15 activités physiques sur l'année, attestées par le flux de données sportives.

01 / Objectifs

Ce que le POC doit prouver

01

Faisabilité

Un pipeline qui tourne de bout en bout, automatisé et reproductible.

02

Données

Identifier et structurer les données — dont des données RH sensibles.

03

Impact financier

Chiffrer le coût, et le recalculer si un paramètre change.

01 / Objectifs

Ce que le POC doit prouver

01

Faisabilité

Un pipeline qui tourne de bout en bout, automatisé et reproductible.

02

Données

Identifier et structurer les données — dont des données RH sensibles.

03

Impact financier

Chiffrer le coût, et le recalculer si un paramètre change.

01 / Objectifs

Ce que le POC doit prouver

01

Faisabilité

Un pipeline qui tourne de bout en bout, automatisé et reproductible.

02

Données

Identifier et structurer les données — dont des données RH sensibles.

03

Impact financier

Chiffrer le coût, et le recalculer si un paramètre change.

01 / Objectifs

Ce que le POC doit prouver

01

Faisabilité

Un pipeline qui tourne de bout en bout, automatisé et reproductible.

02

Données

Identifier et structurer les données — dont des données RH sensibles.

03

Impact financier

Chiffrer le coût, et le recalculer si un paramètre change.

01 / Objectifs

Ce que le POC doit prouver

01

Faisabilité

Un pipeline qui tourne de bout en bout, automatisé et reproductible.

02

Données

Identifier et structurer les données — dont des données RH sensibles.

03

Impact financier

Chiffrer le coût, et le recalculer si un paramètre change.

02

/ Architecture

Un flux orchestré, trois couches de données.

02 / Architecture

Un flux orchestré, trois couches

02 / Architecture

Un flux orchestré, trois couches

SOURCES

02 / Architecture

Un flux orchestré, trois couches

SOURCES

Fichiers RH (XLSX)

161 salariés · PII

02 / Architecture

Un flux orchestré, trois couches

SOURCES

Fichiers RH (XLSX)

161 salariés · PII

Pratiques sportives

déclaratif

02 / Architecture

Un flux orchestré, trois couches

SOURCES

Fichiers RH (XLSX)

161 salariés · PII

Pratiques sportives

déclaratif

Générateur Strava

JSON API-like · 3948

02 / Architecture

Un flux orchestré, trois couches

SOURCES

Fichiers RH (XLSX)

161 salariés · PII

Pratiques sportives

déclaratif

Générateur Strava

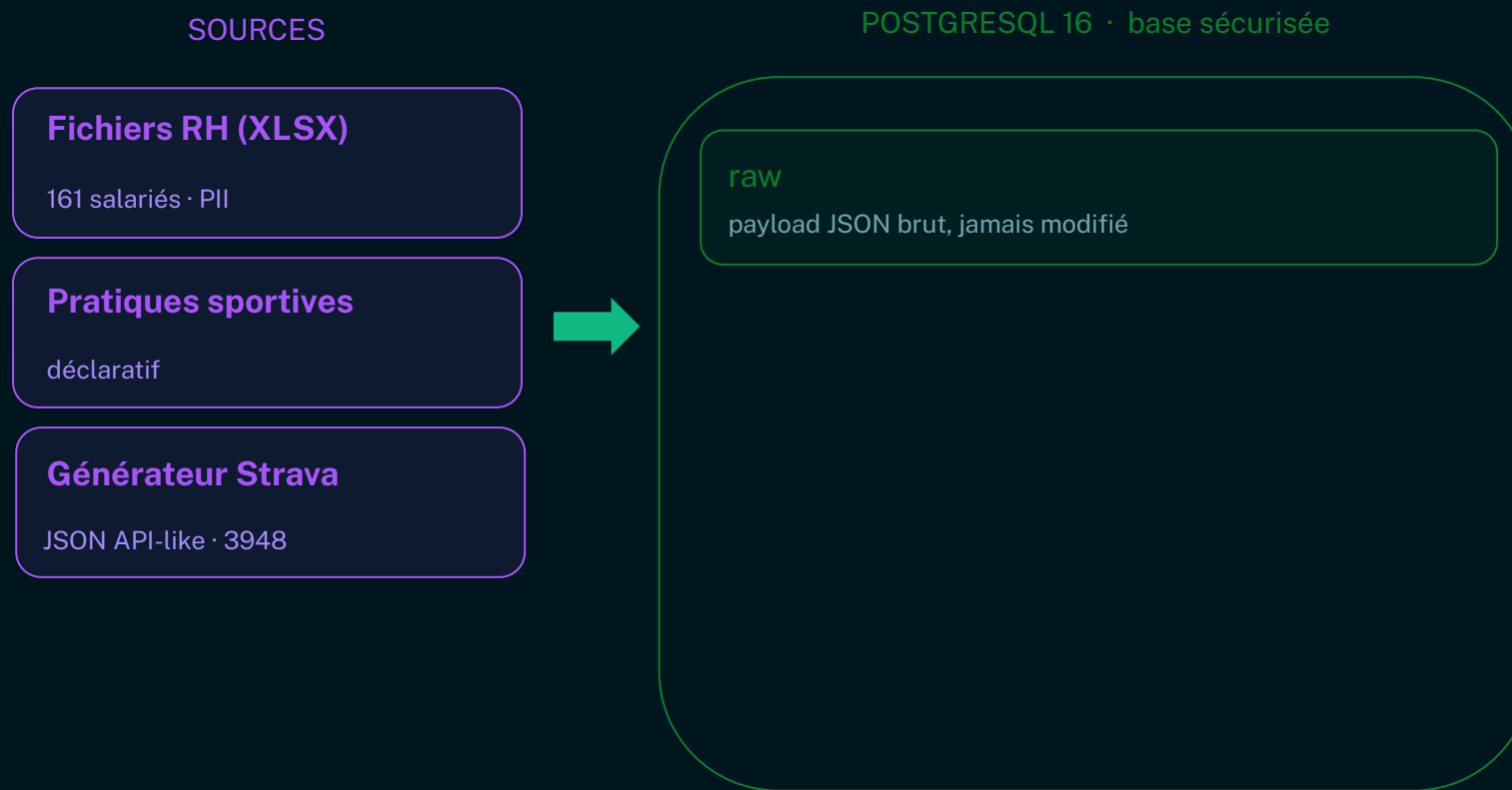
JSON API-like · 3948

POSTGRESQL 16 · base sécurisée



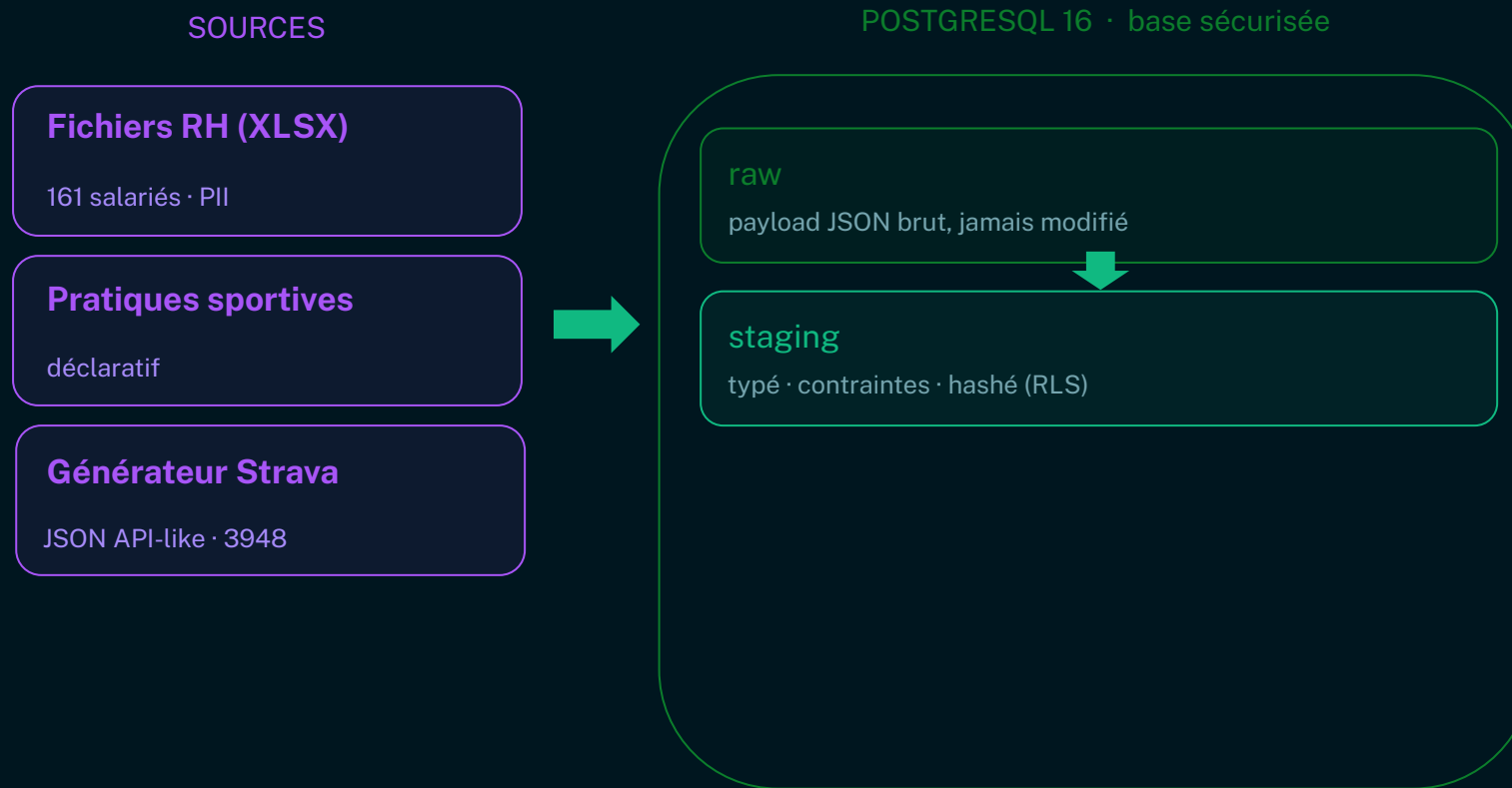
02 / Architecture

Un flux orchestré, trois couches



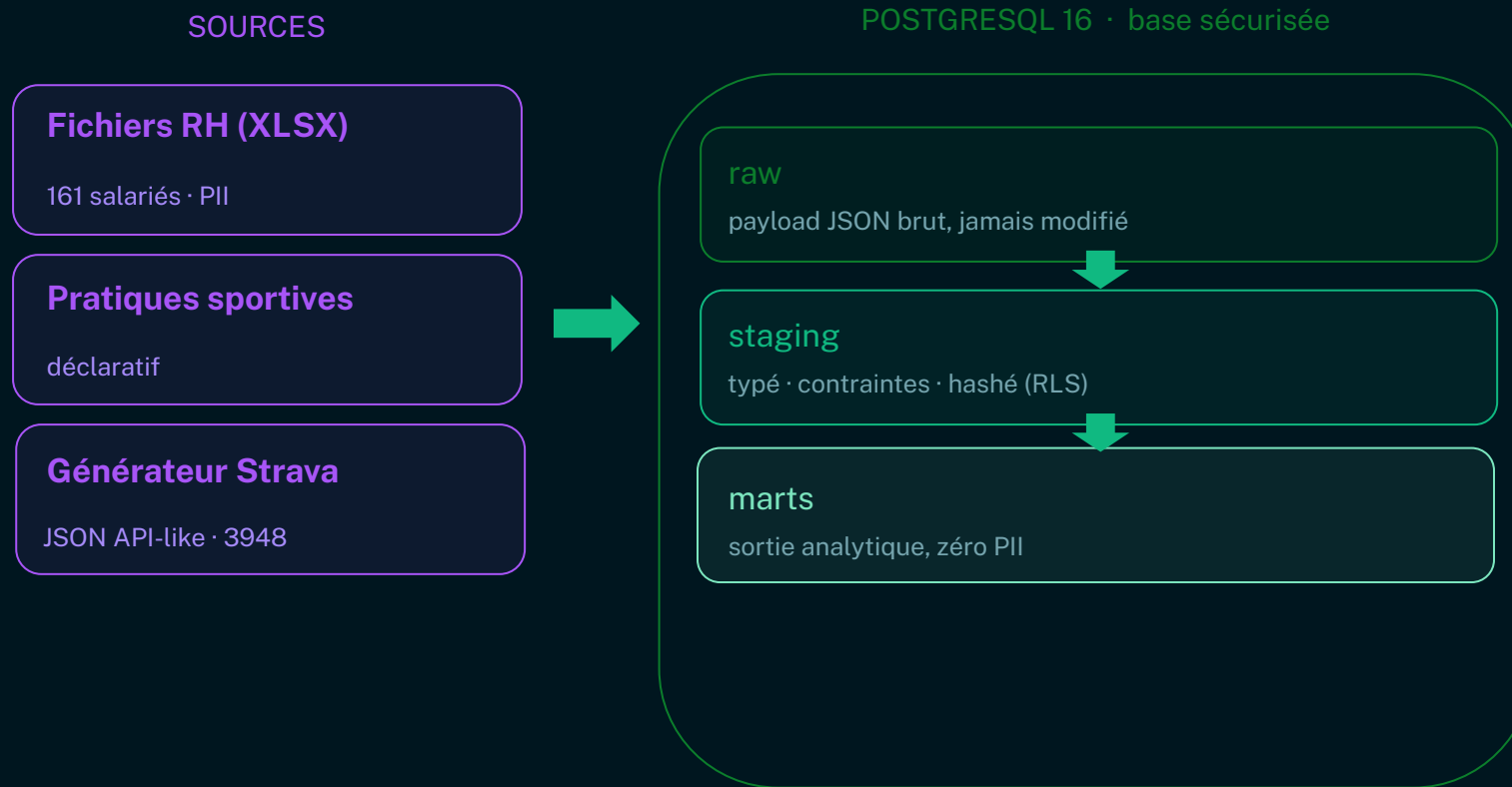
02 / Architecture

Un flux orchestré, trois couches



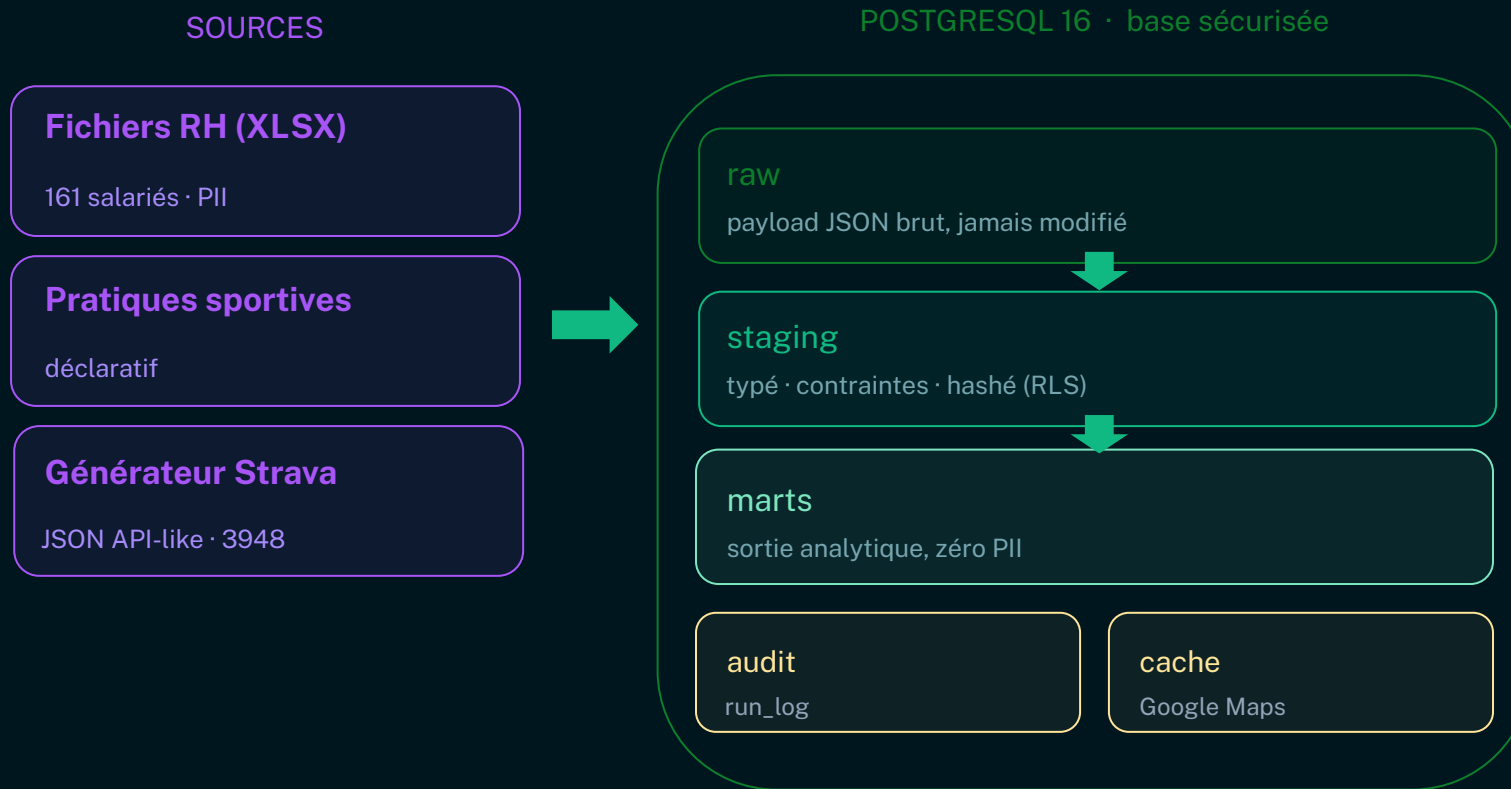
02 / Architecture

Un flux orchestré, trois couches



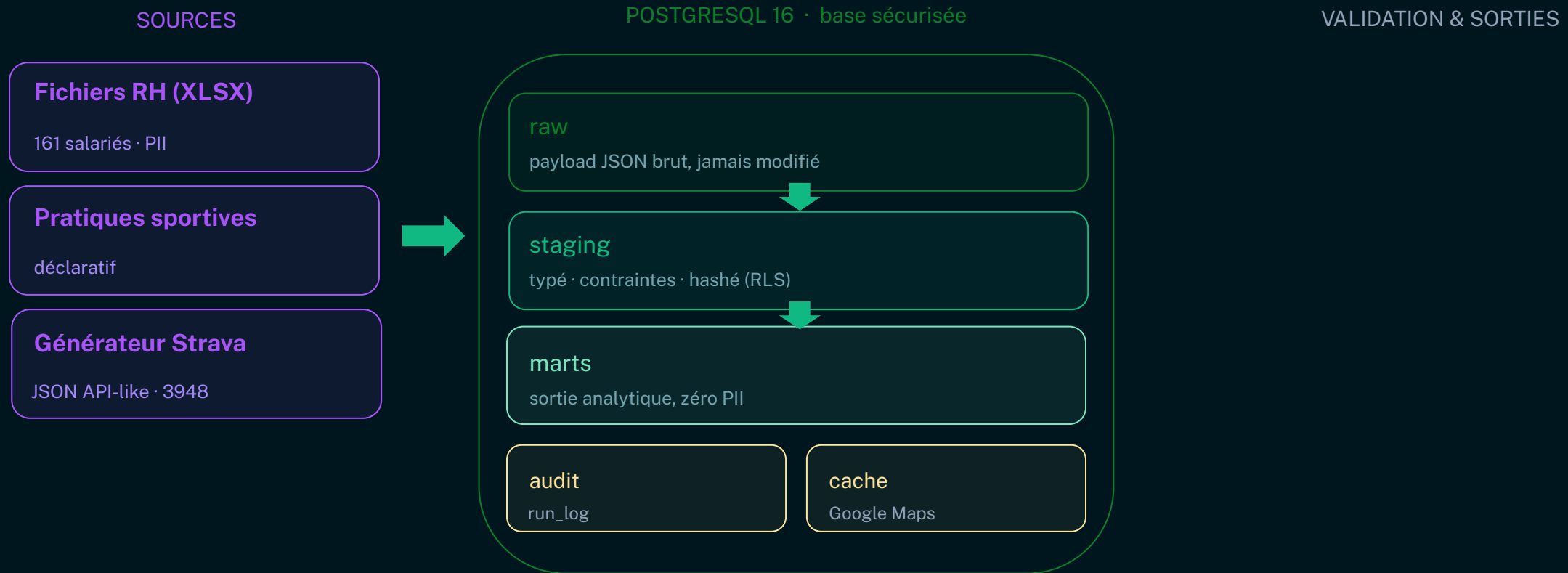
02 / Architecture

Un flux orchestré, trois couches



02 / Architecture

Un flux orchestré, trois couches



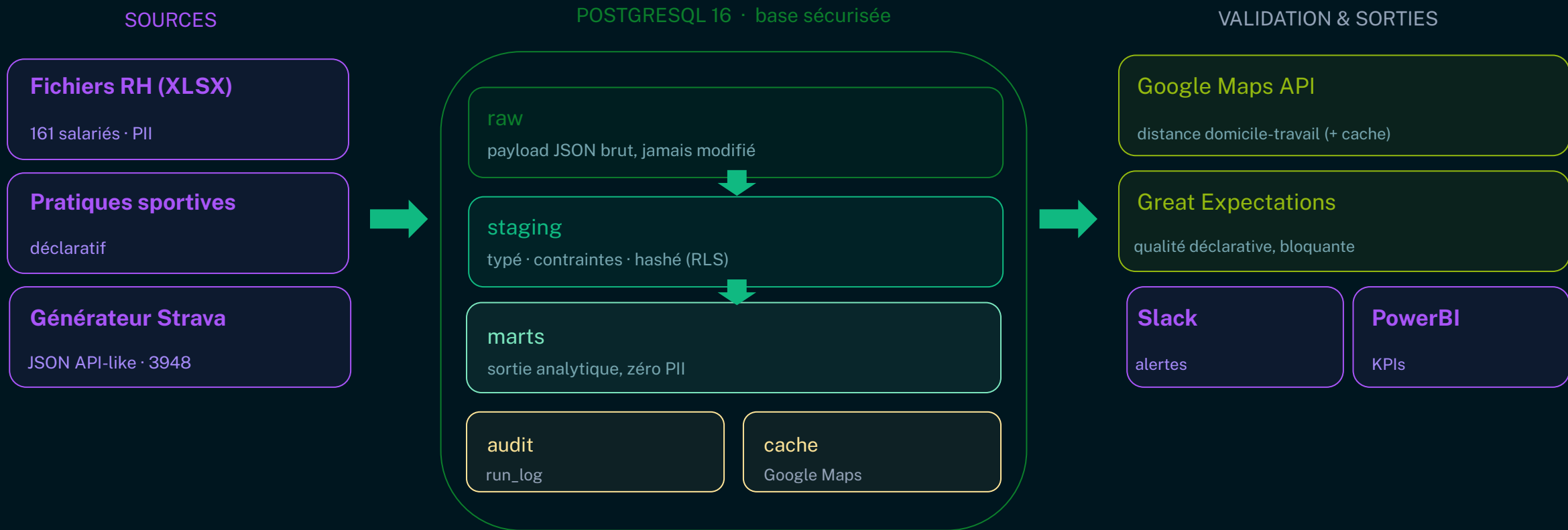
02 / Architecture

Un flux orchestré, trois couches



02 / Architecture

Un flux orchestré, trois couches



02 / Architecture

Un flux orchestré, trois couches



KESTRA orchestre le tout — trigger mensuel, replay, monitoring, alertes

Mise en place

Quatre services, une commande



Mise en place

Quatre services, une commande

Docker Compose



Mise en place

Quatre services, une commande



Docker Compose

sds-kestra:local

Mise en place

Quatre services, une commande



Docker Compose

sds-kestra:local

 Python 3.11

pandas · openpyxl
SQLAlchemy
Great Expectations
requests

Mise en place

Quatre services, une commande



Docker Compose

sds-kestra:local



Python 3.11

pandas · openpyxl
SQLAlchemy
Great Expectations
requests



Web UI

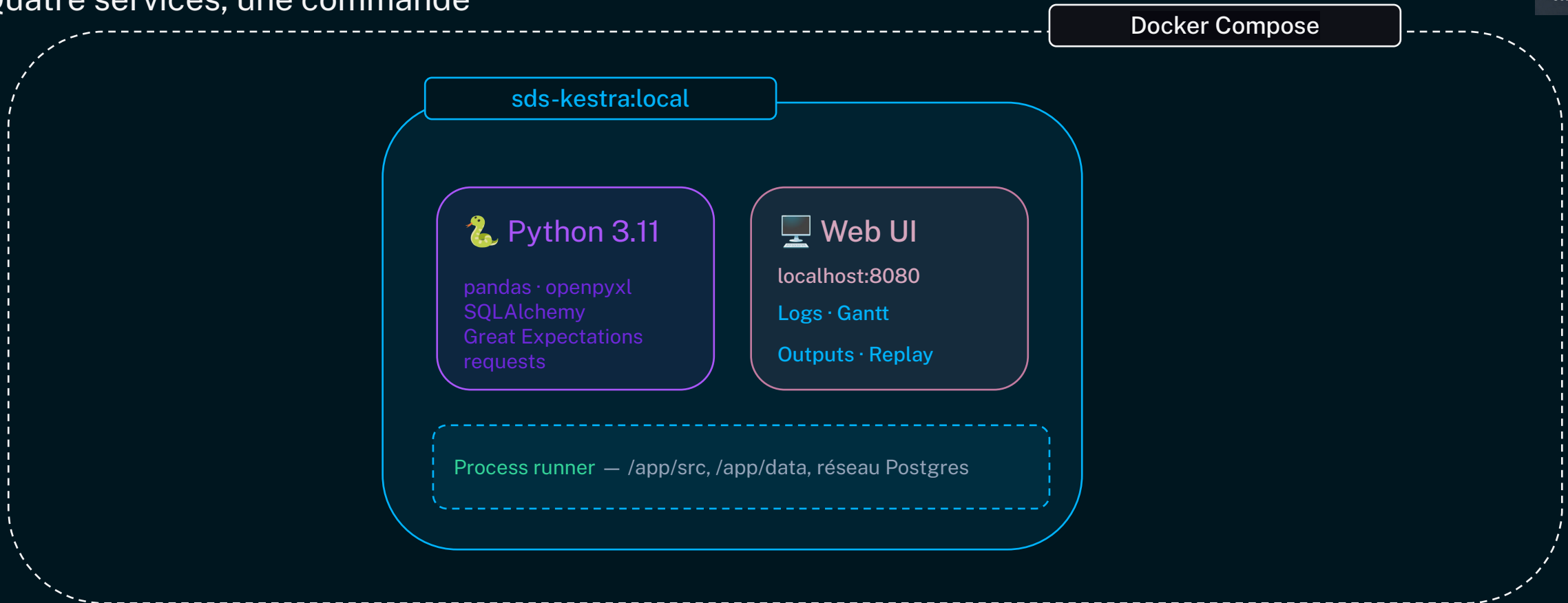
localhost:8080

Logs · Gantt

Outputs · Replay

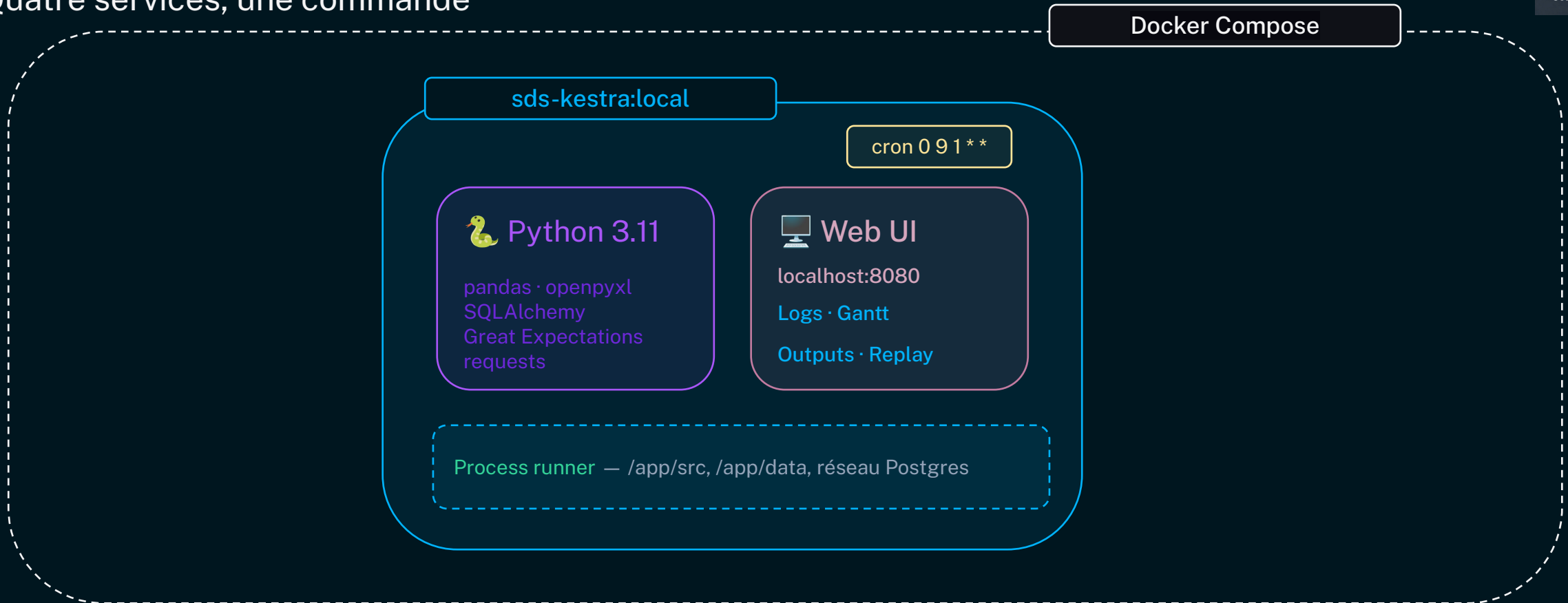
Mise en place

Quatre services, une commande



Mise en place

Quatre services, une commande



Mise en place

Quatre services, une commande



Docker Compose

kestra-init

Service d'amorçage
one-shot (curl).

Importe le flow

sport_advantages_etl.
yaml

via l'API REST Kestra
au démarrage.

sds-kestra:local

cron 0 9 1 * *



Python 3.11

pandas · openpyxl
SQLAlchemy
Great Expectations
requests



Web UI

localhost:8080

Logs · Gantt

Outputs · Replay

Process runner — /app/src, /app/data, réseau Postgres

Mise en place

Quatre services, une commande



Docker Compose

kestra-init

Service d'amorçage
one-shot (curl).

Importe le flow

sport_advantages_etl.
yaml

via l'API REST Kestra
au démarrage.

sds-kestra:local

cron 0 9 1 * *

 Python 3.11

pandas · openpyxl
SQLAlchemy
Great Expectations
requests

 Web UI

localhost:8080

Logs · Gantt

Outputs · Replay

Process runner — /app/src, /app/data, réseau Postgres

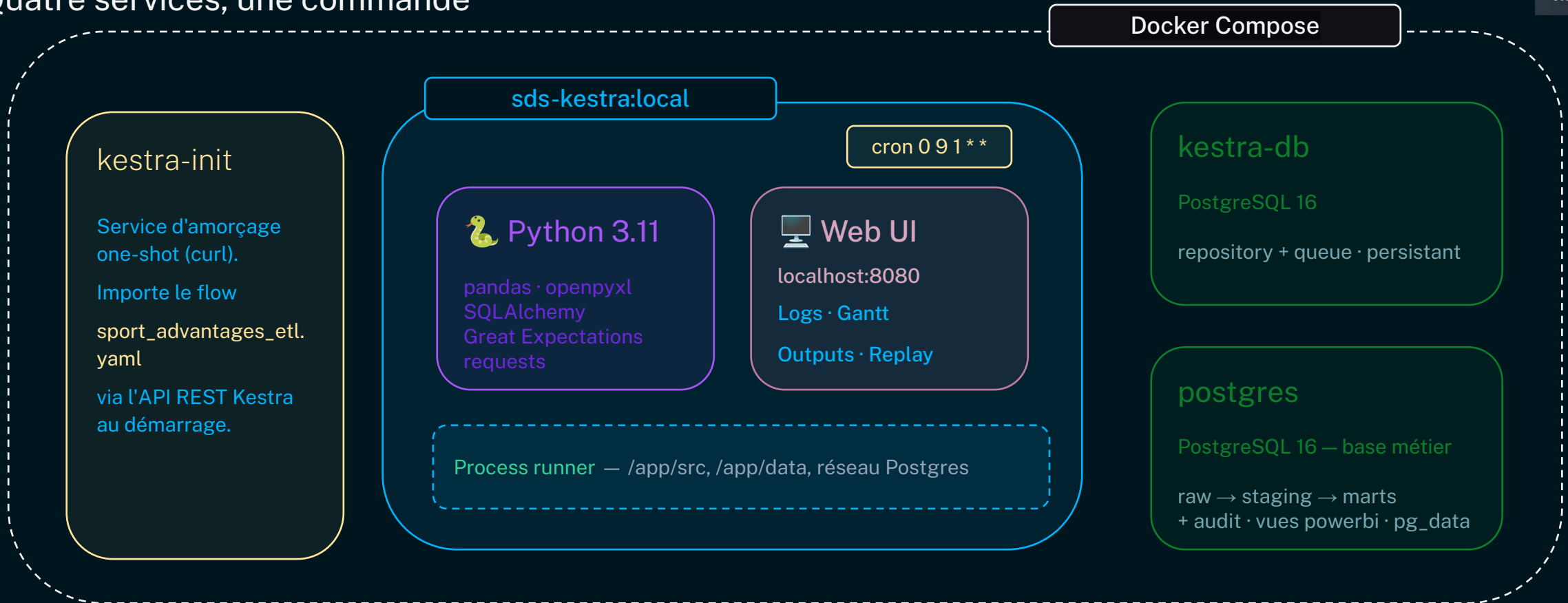
kestra-db

PostgreSQL 16

repository + queue · persistant

Mise en place

Quatre services, une commande



Mise en place

Quatre services, une commande



Docker Compose

kestra-init

Service d'amorçage
one-shot (curl).

Importe le flow

sport_advantages_etl.
yaml

via l'API REST Kestra
au démarrage.

sds-kestra:local

cron 0 9 1 * *



Python 3.11

pandas · openpyxl
SQLAlchemy
Great Expectations
requests



Web UI

localhost:8080

Logs · Gantt

Outputs · Replay

Process runner — /app/src, /app/data, réseau Postgres

kestra-db

PostgreSQL 16

repository + queue · persistant

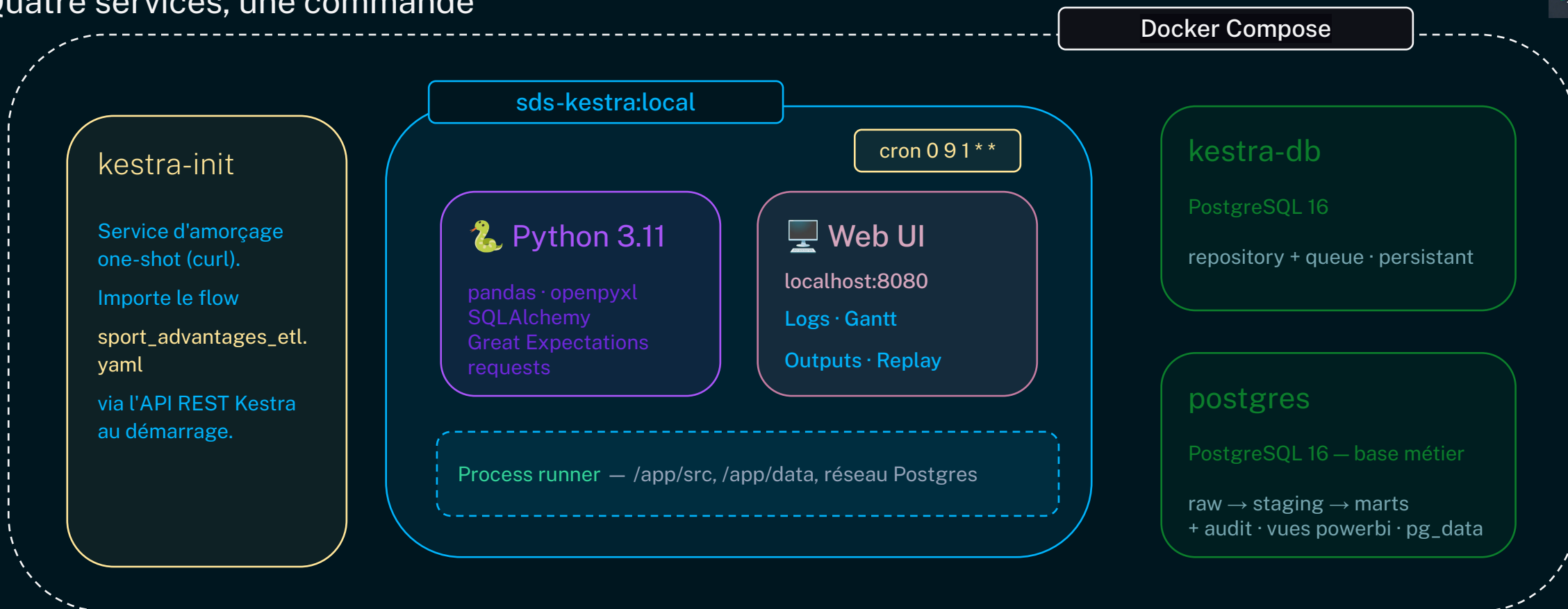
postgres

PostgreSQL 16 — base métier

raw → staging → marts
+ audit · vues powerbi · pg_data

Mise en place

Quatre services, une commande



HORS CONTENEUR

Google Maps API

distance domicile-travail (validate_geo)

Mise en place

Quatre services, une commande



Docker Compose

kestra-init

Service d'amorçage
one-shot (curl).

Importe le flow

sport_advantages_etl.
yaml

via l'API REST Kestra
au démarrage.

sds-kestra:local

cron 0 9 1 * *

 Python 3.11

pandas · openpyxl
SQLAlchemy
Great Expectations
requests

 Web UI

localhost:8080

Logs · Gantt

Outputs · Replay

Process runner — /app/src, /app/data, réseau Postgres

kestra-db

PostgreSQL 16

repository + queue · persistant

postgres

PostgreSQL 16 — base métier

raw → staging → marts
+ audit · vues powerbi · pg_data

HORS CONTENEUR

Google Maps API

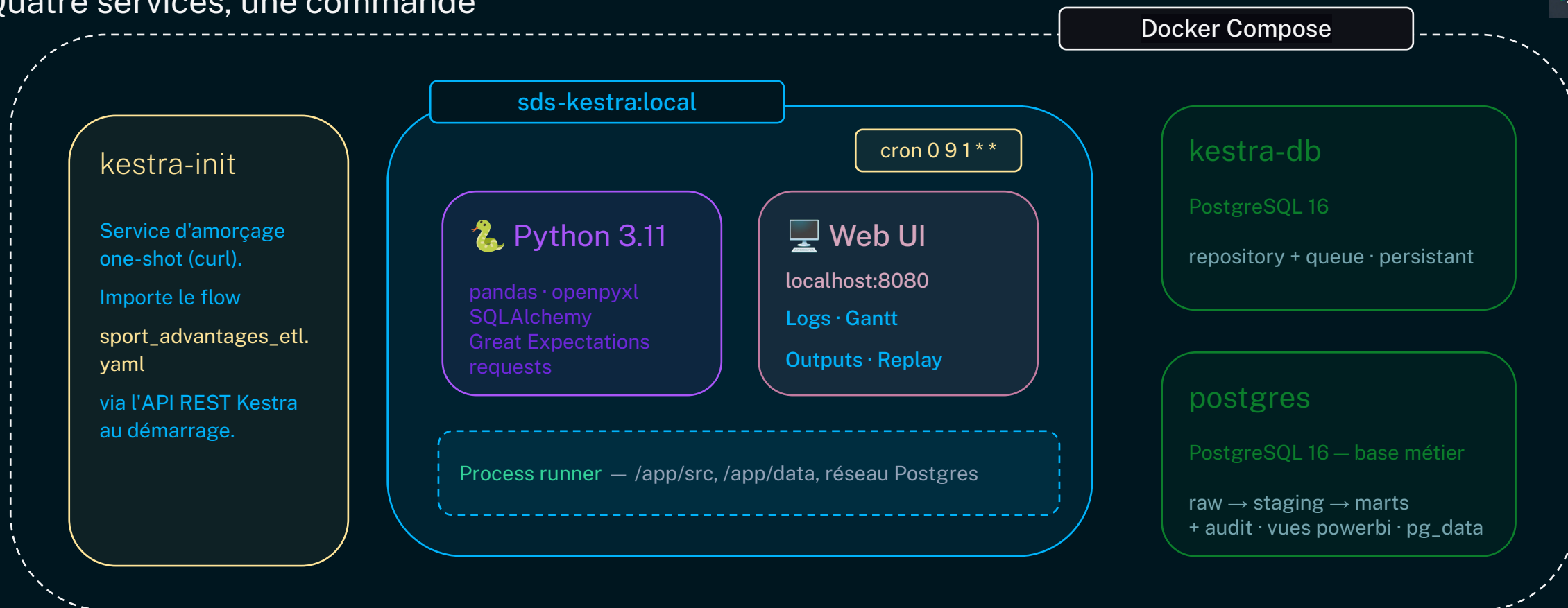
distance domicile-travail (validate_geo)

Slack webhook

notifications + alertes d'échec

Mise en place

Quatre services, une commande



HORS CONTENEUR

Google Maps API

distance domicile-travail (validate_geo)

Slack webhook

notifications + alertes d'échec

Power BI

KPIs — .pbix branché sur les vues

02 / Choix techniques

La bonne brique pour chaque besoin

BRIQUE	OUTIL	POURQUOI
Orchestration	Kestra	Replay de l'historique, monitoring, trigger cron
Base de données	PostgreSQL 16	Rôles, RLS, audit — adapté aux PII RH
Traitement	Python + pandas	Extraction, génération, calculs métier
Qualité	Great Expectations	Tests déclaratifs, bloquants si échec
Géocodage	Google Maps API	Distance réelle domicile-travail (+ cache)
Restitution	PowerBI	KPIs, paramètre de taux dynamique
Conteneurs	Docker Compose	Stack reproductible en une commande

03

/ Pipeline & qualité

Huit étapes, et trois lignes de défense sur la donnée.

03 / Flux ETL

Le flux de traitement, étape par étape

03 / Flux ETL

Le flux de traitement, étape par étape



03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

1

Extract RH

XLSX → raw.employees

161 salariés

03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes



03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

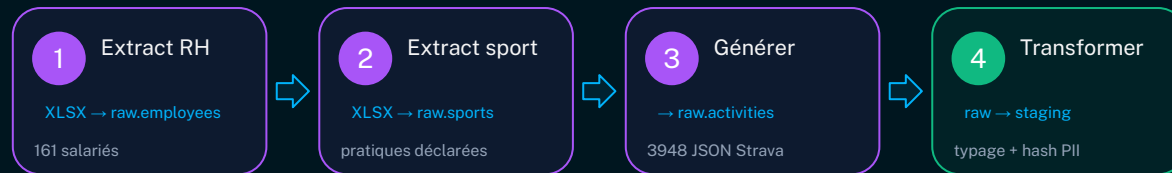


03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

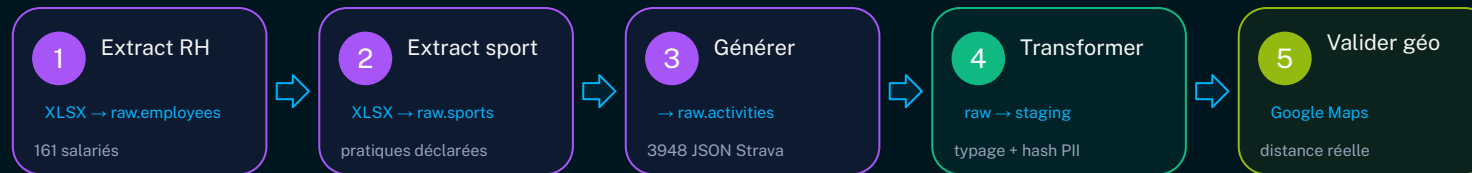


03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes



03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

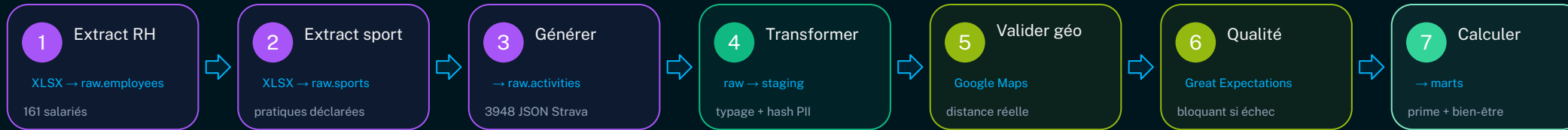


03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

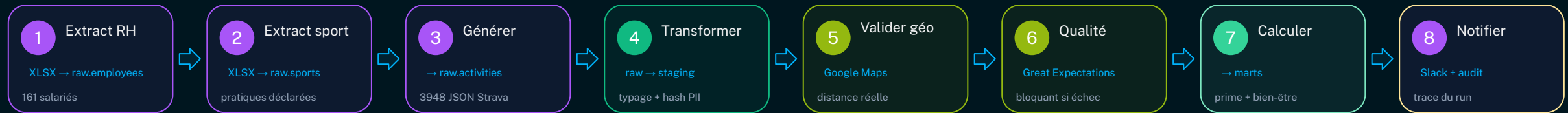


03 / Flux ETL



Le flux de traitement, étape par étape

Pipeline ETL en 8 étapes

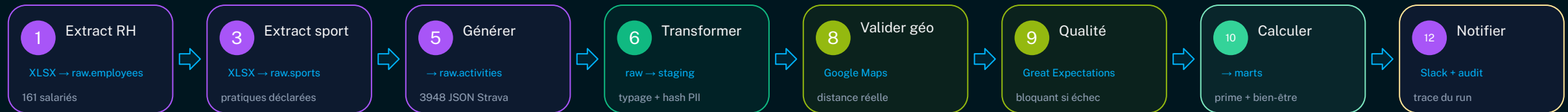


03 / Flux ETL



8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)

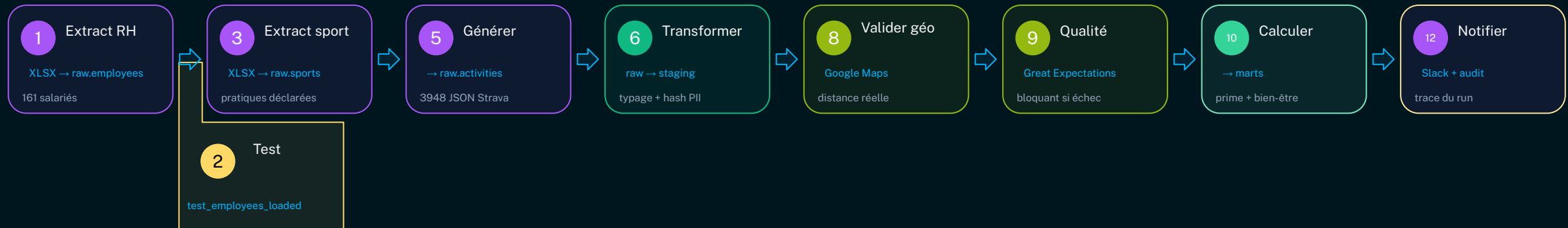


03 / Flux ETL



8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)

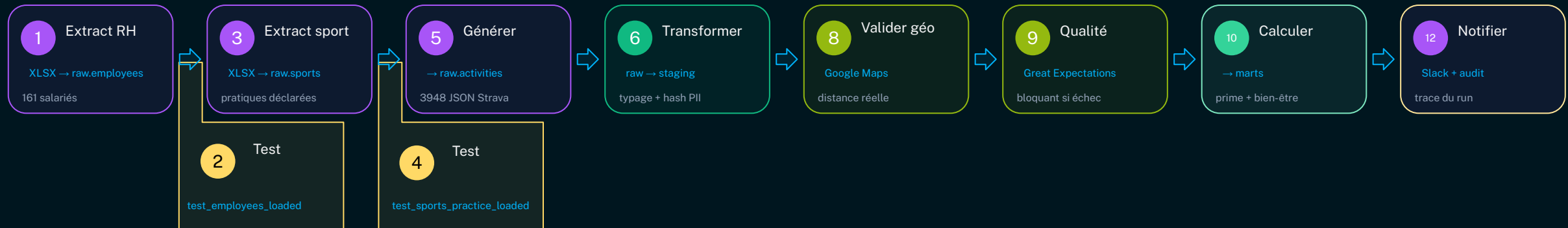


03 / Flux ETL



8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)

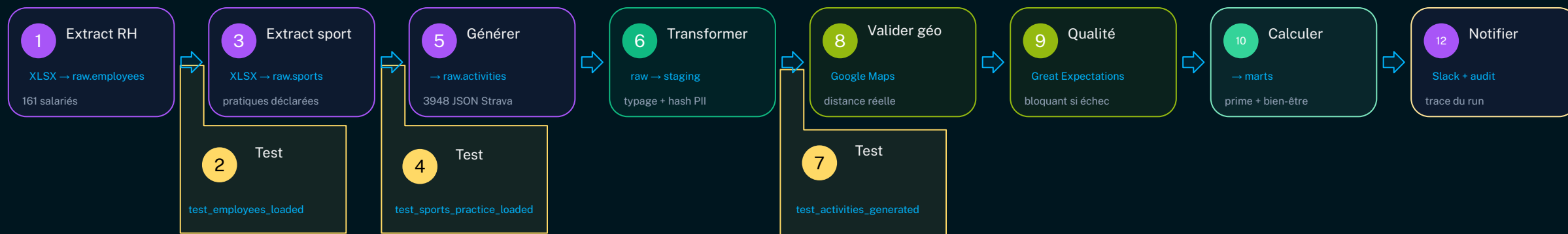


03 / Flux ETL



8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)

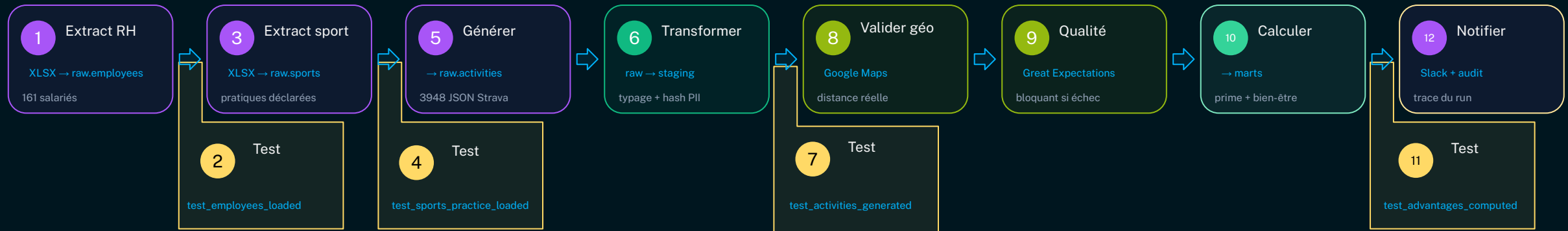


03 / Flux ETL

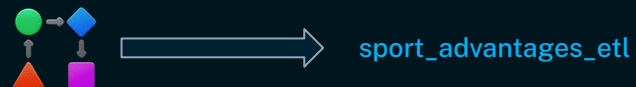


8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)

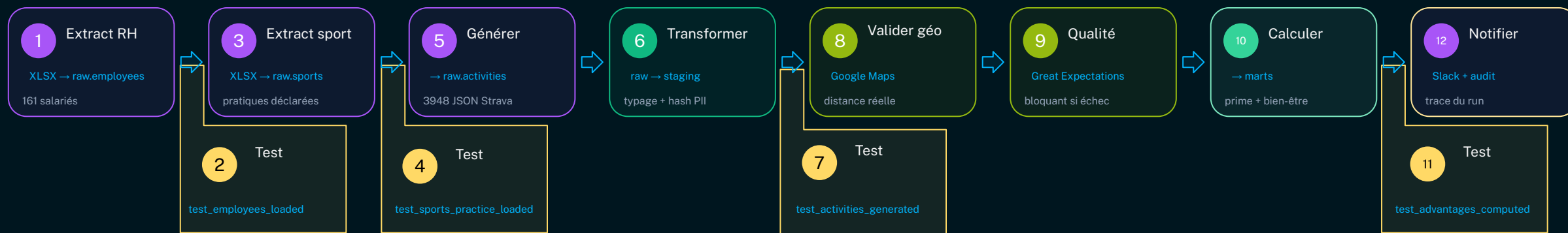


03 / Flux ETL



8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)



Échec d'une étape → le pipeline s'arrête et alerte sur Slack.

Jamais de calcul sur données invalides ; les déclarations suspectes sont calculées mais flaguées pour audit.

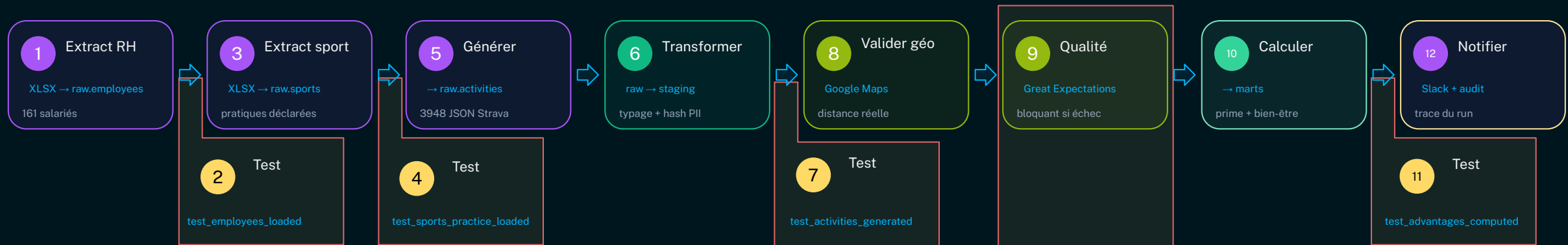
03 / Flux ETL



8 étapes, 12 tâches

Pipeline ETL en 8 étapes, orchestré dans Kestra en 12 tasks : 8 étapes métier + 4 tests-sentinelles bloquants (fail-fast entre chaque phase critique)

Combien d'étapes peuvent stopper le run ? », la vraie réponse est 5 (4 sentinelles + qualité GE)



Échec d'une étape → le pipeline s'arrête et alerte sur Slack.

Jamais de calcul sur données invalides ; les déclarations suspectes sont calculées mais flaguées pour audit.

03 / Qualité

Trois lignes de défense

03 / Qualité

Trois lignes de défense

1

Contraintes SQL

À l'entrée en base : distance ≥ 0 , dates valides, transport dans une liste fermée. Rejet immédiat.

03 / Qualité

Trois lignes de défense

1

Contraintes SQL

À l'entrée en base : distance ≥ 0 , dates valides, transport dans une liste fermée. Rejet immédiat.

2

Validation géographique

Déclaration « marche » mais domicile à 50 km ? Google Maps calcule la distance réelle → anomalie.

03 / Qualité

Trois lignes de défense

1

Contraintes SQL

À l'entrée en base : distance ≥ 0 , dates valides, transport dans une liste fermée. Rejet immédiat.

2

Validation géographique

Déclaration « marche » mais domicile à 50 km ? Google Maps calcule la distance réelle → anomalie.

3

Great Expectations

Règles métier déclaratives, bloquantes. A détecté un vrai bug : des activités datées dans le futur.

03 / Qualité

Robustesse / résilience

Où	Quoi	Config
Kestra — extract_rh, extract_sport	Absorbe une indispo transitoire de Postgres (redémarrage conteneur)	exponential, 3 essais, 10s→2min, facteur 2
Tenacity — geo.py	Appel Google Maps Distance Matrix	3 essais, exp 1–8s
Tenacity — slack.py	Appel API Slack	3 essais, exp 1–4s

Modèle de données

Tables PostgreSQL et leurs relations

PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

RAW

raw.sports_xlsx

id_salarie
sport_pratique

raw.employees_xlsx

id_salarie
nom, prenom
salaire_brut
adresse_domicile
moyen_deplacement

Modèle de données

Tables PostgreSQL et leurs relations

RAW

STAGING



PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

RAW

raw.sports_xlsx	
id_salarie	
sport_pratique	

raw.employees_xlsx	
id_salarie	
nom, prenom	
salaire_brut	
adresse_domicile	
moyen_deplacement	

STAGING

staging.sports_practice	
PK-FK id_employe	
PK sport_pratique	

staging.employees	
PK id_employe	
nom_hash	
salaire_brut	
distance_domicile_m	
is_declaration_suspect	

Générateur

PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

sports_practice + employees → générateur d'activités → raw.activities (payload Strava-like)

RAW

raw.sports_xlsx

id_salarie
sport_pratique

raw.employees_xlsx

id_salarie
nom, prenom
salaire_brut
adresse_domicile
moyen_deplacement

raw.activities

PK activity_id
athlete_id
payload JSONB

STAGING

staging.sports_practice

PK-FK id_employe
PK sport_pratique

staging.employees

PK id_employe
nom_hash
salaire_brut
distance_domicile_m
is_declaration_suspect

Générateur

PK clé primaire **FK** clé étrangère — référence --- alimentation (raw → staging)

Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

sports_practice + employees → générateur d'activités → raw.activities (payload Strava-like)

RAW

raw.sports_xlsx	
id_salarie	
sport_pratique	

raw.employees_xlsx	
id_salarie	
nom, prenom	
salaire_brut	
adresse_domicile	
moyen_deplacement	

raw.activities	
PK activity_id	
athlete_id	
payload JSONB	

STAGING

staging.sports_practice	
PK-FK id_employee	
PK sport_pratique	

staging.employees	
PK id_employee	
nom_hash	
salaire_brut	
distance_domicile_m	
is_declaration_suspect	

staging.activities	
PK id_activity	
FK source_activity_id	
FK id_employee	
sport_type, distance_m	

Générateur

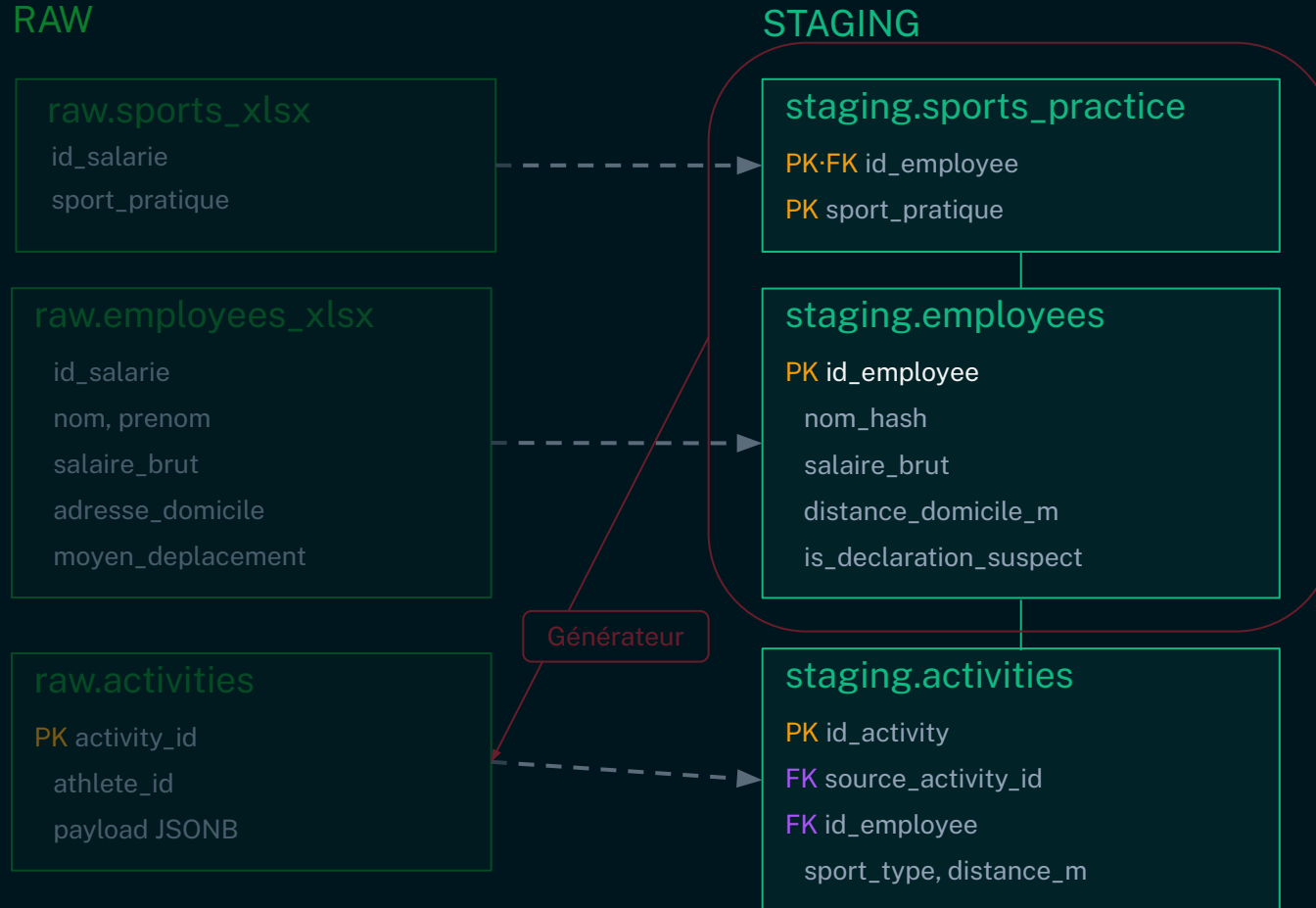
PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

sports_practice + employees → générateur d'activités → raw.activities (payload Strava-like)

RAW

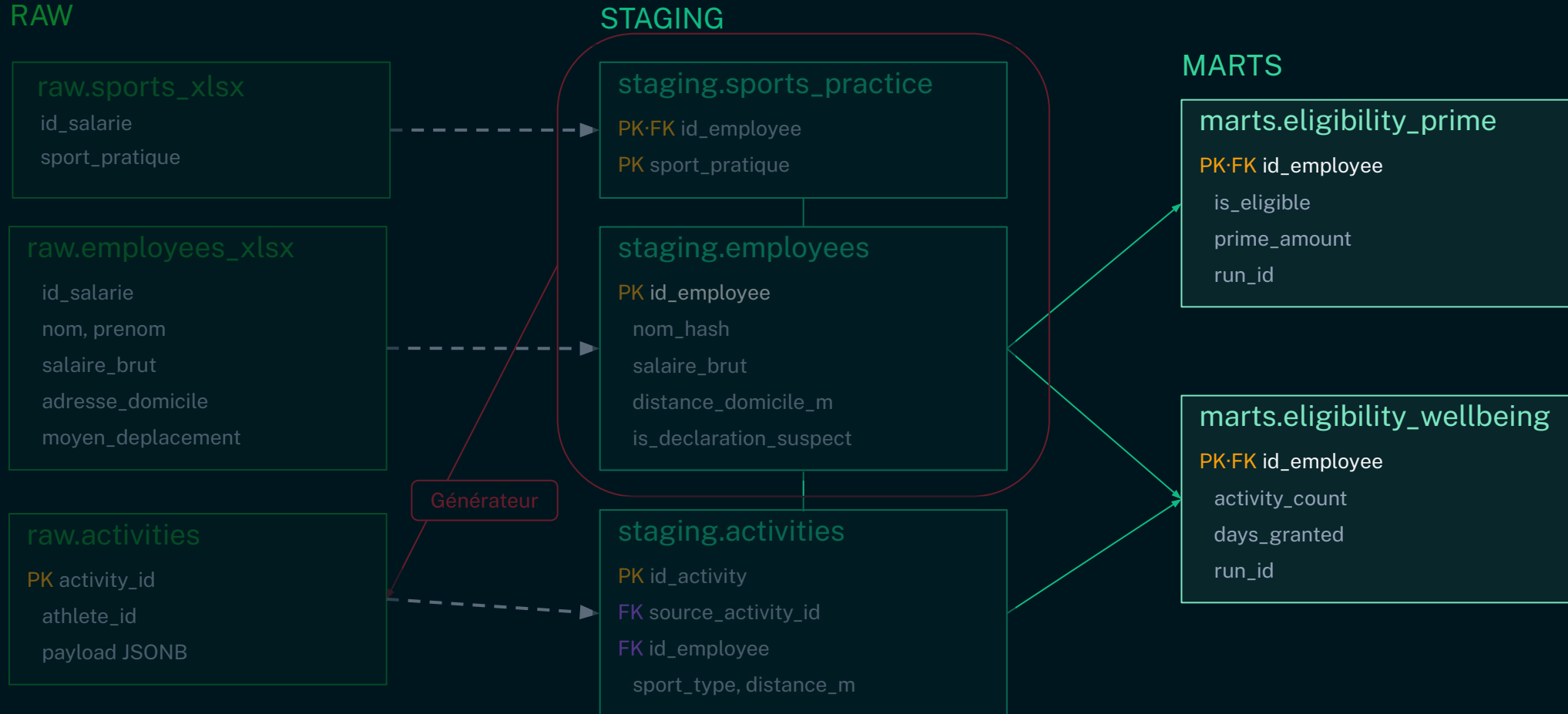


PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

sports_practice + employees → générateur d'activités → raw.activities (payload Strava-like)



PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

sports_practice + employees → générateur d'activités → raw.activities (payload Strava-like)

RAW

raw.sports_xlsx	
id_salarie	
sport_pratique	

raw.employees_xlsx	
id_salarie	
nom, prenom	
salaire_brut	
adresse_domicile	
moyen_deplacement	

raw.activities	
PK activity_id	
athlete_id	
payload JSONB	

STAGING

staging.sports_practice	
PK-FK id_employee	
PK sport_pratique	

staging.employees	
PK id_employee	
nom_hash	
salaire_brut	
distance_domicile_m	
is_declaration_suspect	

staging.activities	
PK id_activity	
FK source_activity_id	
FK id_employee	
sport_type, distance_m	

Générateur

MARTS

marts.eligibility_prime	
PK-FK id_employee	
is_eligible	
prime_amount	
run_id	

marts.eligibility_wellbeing	
PK-FK id_employee	
activity_count	
days_granted	
run_id	

AUDIT / CACHE

audit.run_log	
run_id	
step_name	
status	

cache.gmaps	
PK address_hash	
PK mode	
distance_m	

PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

sports_practice + employees → générateur d'activités → raw.activities (payload Strava-like)

RAW

raw.sports_xlsx
id_salarie
sport_pratique

raw.employees_xlsx
id_salarie
nom, prenom
salaire_brut
adresse_domicile
moyen_deplacement

raw.activities
PK activity_id
athlete_id
payload JSONB

STAGING

staging.sports_practice
PK-FK id_employee
PK sport_pratique

staging.employees
PK id_employee
nom_hash
salaire_brut
distance_domicile_m
is_declaration_suspect

staging.activities
PK id_activity
FK source_activity_id
FK id_employee
sport_type, distance_m

Générateur

MARTS

marts.eligibility_prime
PK-FK id_employee
is_eligible
prime_amount
run_id

marts.eligibility_wellbeing
PK-FK id_employee
activity_count
days_granted
run_id

AUDIT / CACHE

audit.run_log
run_id
step_name
status

cache.gmaps
PK address_hash
PK mode
distance_m

PK clé primaire **FK** clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

Modèle de données

Tables PostgreSQL et leurs relations

Normalisé en entrée (raw/staging) pour la qualité et le RGPD, dénormalisé en sortie (marts) en modèle dimensionnel pour la performance de lecture PowerBI

RAW

raw.sports_xlsx	
id_salarie	
sport_pratique	

raw.employees_xlsx	
id_salarie	
nom, prenom	
salaire_brut	
adresse_domicile	
moyen_deplacement	

raw.activities	
PK activity_id	
athlete_id	
payload JSONB	

STAGING

staging.sports_practice	
PK-FK id_employee	
PK sport_pratique	

staging.employees	
PK id_employee	
nom_hash	
salaire_brut	
distance_domicile_m	
is_declaration_suspect	

staging.activities	
PK id_activity	
FK source_activity_id	
FK id_employee	
sport_type, distance_m	

MARTS

marts.eligibility_prime	
PK-FK id_employee	
is_eligible	
prime_amount	
run_id	

marts.eligibility_wellbeing	
PK-FK id_employee	
activity_count	
days_granted	
run_id	

AUDIT / CACHE

audit.run_log	
run_id	
step_name	
status	

cache.gmaps	
PK address_hash	
PK mode	
distance_m	

★ table de faits ◆ dimension conforme run_id + computed_at = dim. dégénérées vue marts.kpi_by_bu = jointure étoile

PK clé primaire FK clé étrangère — référence --- alimentation (raw → staging)
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

04

/ Sécurité & données personnelles

Des données RH : cloisonnées, pseudonymisées, tracées.

04 / Sécurité et RGPD

Des données RH, donc cloisonnées

04 / Sécurité et RGPD

Des données RH, donc cloisonnées

Mesures en place

3 rôles PostgreSQL : l'analyste ne voit jamais les noms ni salaires en clair

Pseudonymisation : hash salé des identités, salaires en tranches

Row Level Security + triggers d'audit (traçabilité RGPD)

Aucun secret dans le code — variables d'environnement, .gitignore strict

04 / Sécurité et RGPD

Des données RH, donc cloisonnées

Mesures en place

3 rôles PostgreSQL : l'analyste ne voit jamais les noms ni salaires en clair

Pseudonymisation : hash salé des identités, salaires en tranches

Row Level Security + triggers d'audit (traçabilité RGPD)

Aucun secret dans le code — variables d'environnement, .gitignore strict

Audit de sécurité

8 / 9

vulnérabilités corrigées

Bandit : 0 faille critique

Socket Docker retiré

Dépendances à jour

04 / Données personnelles

Le cycle de vie d'une donnée RH

04 / Données personnelles

Le cycle de vie d'une donnée RH

La même donnée, trois niveaux d'exposition selon la couche.

04 / Données personnelles

Le cycle de vie d'une donnée RH

La même donnée, trois niveaux d'exposition selon la couche.

RAW

Reflet brut du fichier RH. Nom, salaire, adresse en clair. Pipeline seul.

Le Gall · 42 000 € · Lattes

04 / Données personnelles

Le cycle de vie d'une donnée RH

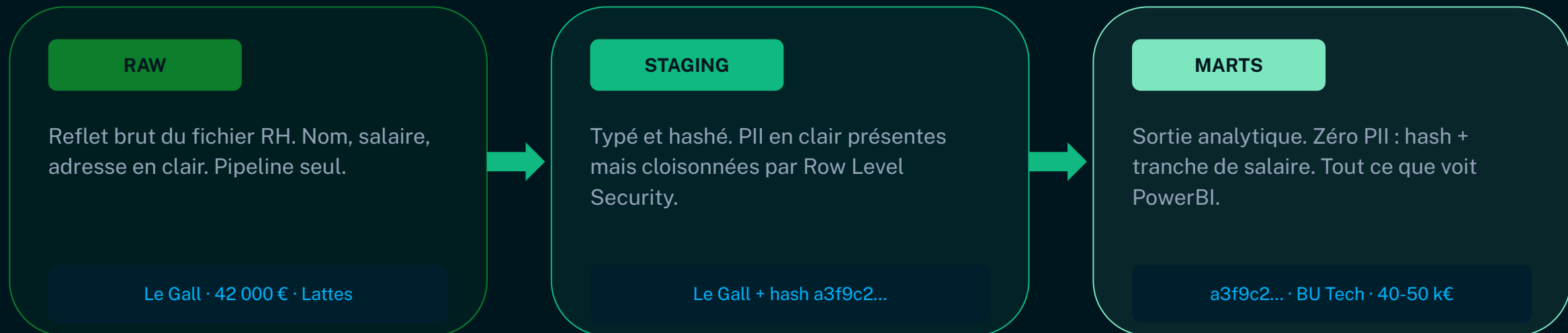
La même donnée, trois niveaux d'exposition selon la couche.



04 / Données personnelles

Le cycle de vie d'une donnée RH

La même donnée, trois niveaux d'exposition selon la couche.



04 / Données personnelles

Le cycle de vie d'une donnée RH

La même donnée, trois niveaux d'exposition selon la couche.



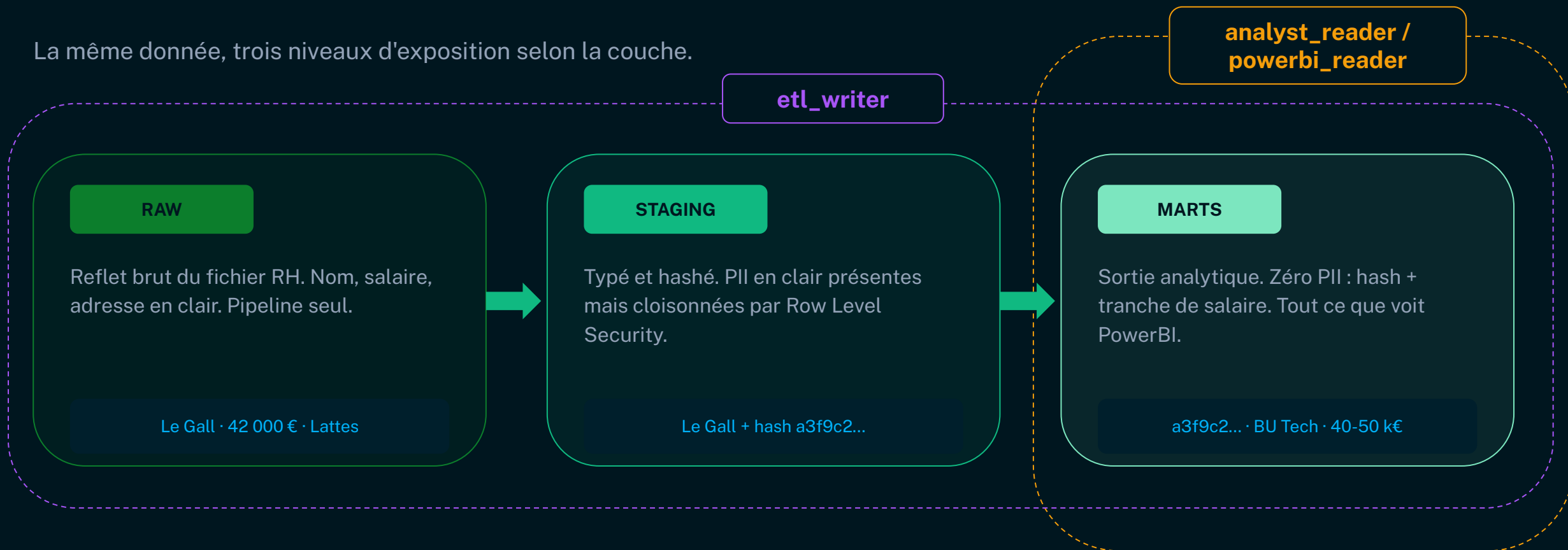
etl_writer écrit/lit les PII · **analyst_reader / powerbi_reader** lisent marts — jamais de PII

Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

04 / Données personnelles

Le cycle de vie d'une donnée RH

La même donnée, trois niveaux d'exposition selon la couche.



etl_writer écrit/lit les PII · **analyst_reader / powerbi_reader** lisent marts — jamais de PII

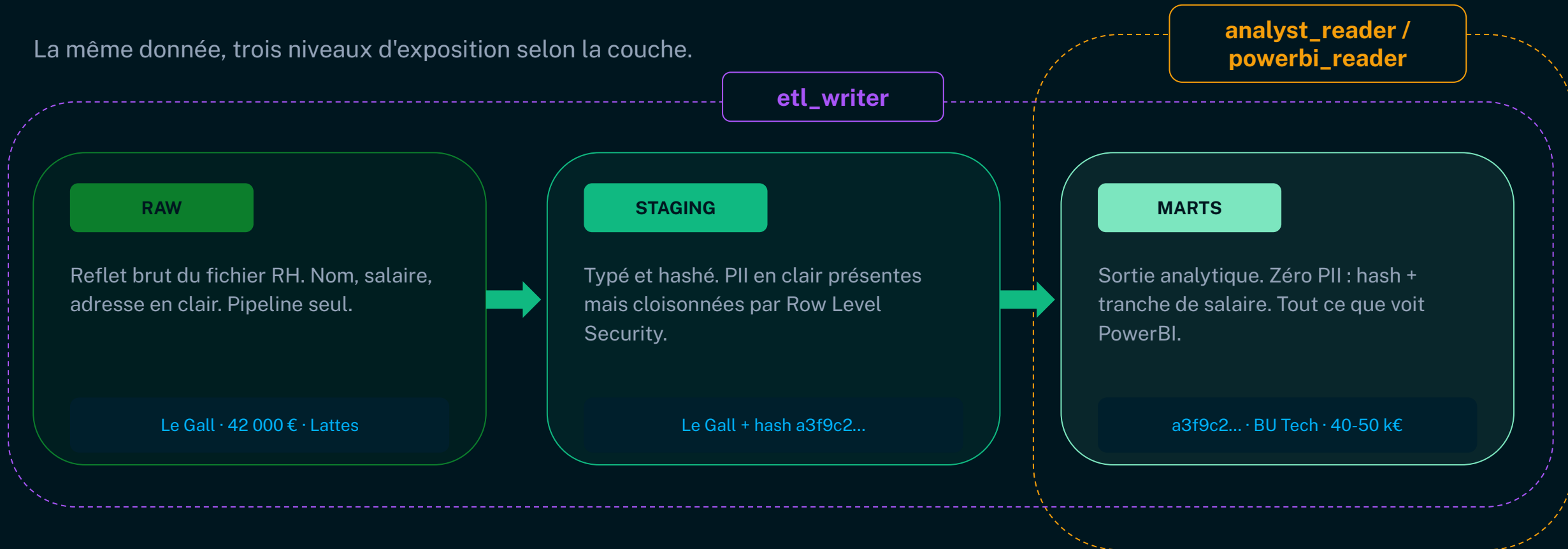
Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

04 / Données personnelles

etl_writer écrit sur toute la chaîne, marts compris, puisque c'est lui qui alimente la couche de restitution. Les lecteurs, eux, ne font que lire marts. marts est le seul schéma partagé : écrit par l'ETL, lu par PowerBI.

Le cycle de vie d'une donnée RH

La même donnée, trois niveaux d'exposition selon la couche.



etl_writer écrit/lit les PII · *analyst_reader / powerbi_reader* lisent marts — jamais de PII

Schémas raw / staging / marts + audit & cache — lignage tracé de la source à la restitution

05

/ Monitoring & résultats

Chaque run tracé ; l'impact financier chiffré.

05 / Monitoring

Chaque exécution est tracée

05 / Monitoring

Chaque exécution est tracée

Dernier run Kestra

SUCCESS

12 / 12 tâches · ~20 s

Volumétrie + durée par étape

Alerte Slack automatique si échec

Tests visibles + replay

05 / Monitoring

Chaque exécution est tracée

Dernier run Kestra

SUCCESS

12 / 12 tâches · ~20 s

Volumétrie + durée par étape
Alerte Slack automatique si échec
Tests visibles + replay

audit.run_log (extrait)

ÉTAPE	LIGNES	ST
extract_rh	161	OK
generate_activities	3948	OK
validate_quality	3948	OK
compute_advantages	68	OK
post_slack	10	OK

05 / Résultats

Ce que coûtent les avantages

Tous ces chiffres se recalculent en un clic si le taux de prime change.

05 / Résultats

Ce que coûtent les avantages

172 482 €

coût annuel de la prime

68 / 161

éligibles à la prime

0

déclaration suspecte

Tous ces chiffres se recalculent en un clic si le taux de prime change.

05 / Résultats

Ce que coûtent les avantages

172 482 €

coût annuel de la prime

68 / 161

éligibles à la prime

0

déclaration suspecte

44

éligibles bien-être

220

jours bien-être accordés

3 948

activités analysées

Tous ces chiffres se recalculent en un clic si le taux de prime change.

06

/ Démo & industrialisation

La démo live, puis le chemin vers la production.

06 / Démonstration

Deux scénarios en direct

Scénario A

Changer le taux de prime

1. Passer le taux de 5 % à 7 % dans Kestra
2. Relancer le pipeline
3. Le coût total évolue dans PowerBI

Scénario B

Insérer une activité live

1. Ajouter une nouvelle course
2. Le message arrive dans Slack
3. L'activité apparaît dans le reporting

06 / Démonstration

Deux scénarios en direct

Scénario A

Changer le taux de prime

1. Passer le taux de 5 % à 7 % dans Kestra
2. Relancer le pipeline
3. Le coût total évolue dans PowerBI

Scénario B

Insérer une activité live

1. Ajouter une nouvelle course
2. Le message arrive dans Slack
3. L'activité apparaît dans le reporting

06 / Démonstration

Deux scénarios en direct

Scénario A

Changer le taux de prime

1. Passer le taux de 5 % à 7 % dans Kestra
2. Relancer le pipeline
3. Le coût total évolue dans PowerBI

Scénario B

Insérer une activité live

1. Ajouter une nouvelle course
2. Le message arrive dans Slack
3. L'activité apparaît dans le reporting

06 / Démonstration

Deux scénarios en direct

Scénario A

Changer le taux de prime

1. Passer le taux de 5 % à 7 % dans Kestra
2. Relancer le pipeline
3. Le coût total évolue dans PowerBI

Scénario B

Insérer une activité live

1. Ajouter une nouvelle course
2. Le message arrive dans Slack
3. L'activité apparaît dans le reporting

06 / Industrialisation

Pensé pour passer en production

06 / Industrialisation

Pensé pour passer en production

Déjà en place

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

Idempotent

Rejouable sans doublons (Slack, calculs)

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

Idempotent

Rejouable sans doublons (Slack, calculs)

Observable

Tests visibles, alerte Slack, rapport GE exporté

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

Idempotent

Rejouable sans doublons (Slack, calculs)

Observable

Tests visibles, alerte Slack, rapport GE exporté

Prochaines étapes

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

Idempotent

Rejouable sans doublons (Slack, calculs)

Observable

Tests visibles, alerte Slack, rapport GE exporté

Prochaines étapes

API Strava réelle

Remplacer le générateur par les vraies données

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

Idempotent

Rejouable sans doublons (Slack, calculs)

Observable

Tests visibles, alerte Slack, rapport GE exporté

Prochaines étapes

API Strava réelle

Remplacer le générateur par les vraies données

CI/CD

Tests + audit sécurité à chaque commit

06 / Industrialisation

Pensé pour passer en production

Déjà en place

Reproductible

Stack conteneurisée, démarrage en une commande

Idempotent

Rejouable sans doublons (Slack, calculs)

Observable

Tests visibles, alerte Slack, rapport GE exporté

Prochaines étapes

API Strava réelle

Remplacer le générateur par les vraies données

CI/CD

Tests + audit sécurité à chaque commit

Secret manager

Rotation des secrets en production

Conclusion

Le POC valide les trois objectifs.

Conclusion

Le POC valide les trois objectifs.

✓ Faisable : pipeline complet, vert, en ~20 secondes

Conclusion

Le POC valide les trois objectifs.

- ✓ Faisable : pipeline complet, vert, en ~20 secondes
- ✓ Données maîtrisées : sensibles, cloisonnées, tracées

Conclusion

Le POC valide les trois objectifs.

- ✓ Faisable : pipeline complet, vert, en ~20 secondes
- ✓ Données maîtrisées : sensibles, cloisonnées, tracées
- ✓ Coût chiffré et pilotable : 172 482 € / an, recalculable à la demande

Conclusion

Le POC valide les trois objectifs.

- ✓ Faisable : pipeline complet, vert, en ~20 secondes
- ✓ Données maîtrisées : sensibles, cloisonnées, tracées
- ✓ Coût chiffré et pilotable : 172 482 € / an, recalculable à la demande

Recommandation : industrialiser, en commençant par l'intégration Strava.

Merci.

Place à la discussion et à la démonstration.

Morgan Le Gall — POC Avantages Sportifs — Sport Data Solution

